

Karbowanec PQ Base Layer Proposal

Transaction Family v3 — KEM-Derived Spend Keys, Fully Specified

Community review draft — all critical parameters normative

v2.3 — KDSK naming, denomination-membership framing, shared KEM session

Recommended Phase-1 cryptographic suite

Spend authorization:	ML-DSA-65, per-output KEM-derived subkeys
Receiver scanning:	ML-KEM-768 (replaces CryptoNote ECDH)
Hash / KDF:	SHA3-256 + HKDF-SHA3-256 (salt=0x00*32, L=32)
AEAD cipher:	ChaCha20-Poly1305 (IETF, 96-bit nonce)
Payload:	Encrypts rho only (32B); amount bound into AEAD tag via AAD
Amounts:	Plain transparent amounts (quantum-safe)
Double-spend:	Hash nullifiers computed by node, not serialized
Privacy:	Stealth receiving; ring sigs deferred to Phase 2
rho:	32 uniform random bytes, generated by sender per output

1. Scope and Position

This proposal defines a Phase-1 post-quantum transaction family for Karbowanec. It solves ownership, receiving, and spending first, without attempting to ship research-grade post-quantum RingCT in the same fork. Every parameter in this document is normative. Implementations must not deviate from the specified hash inputs, key sizes, or serialization order.

Per-output spend keys are derived directly from the ML-KEM shared secret established during output creation. This eliminates the Merkle auth tree and epoch system of the prior draft, removes address exhaustion concerns, and reduces implementation surface to well-understood HKDF derivation.

- Standardized NIST PQC primitives only — ML-KEM-768 (FIPS 203) and ML-DSA-65 (FIPS 204).
- One clean address family for new PQ wallets. Address is static; spend keys are one-time.
- No elliptic-curve operations anywhere in the new transaction validation path.
- Unlimited outputs to the same address — no epoch rotation, no exhaustion.
- Nullifiers are computed by the node from revealed fields; they are not serialized in transactions.

Known privacy property — spend linkability. Receiving is private: the stealth address system hides which address an output belongs to. Spending is publicly linkable: when an output is spent, the input reveals `auth_pub` and `rho_reveal` in plaintext. Anyone can recompute `spend_commit = SHA3-256(domain || auth_pub || rho_reveal)` and match the spending input to the exact earlier output. This means the link between a received output and its spend transaction is visible on chain. This is a deliberate Phase-1 trade-off, not an oversight. Spend-side privacy requires post-quantum ring signatures or equivalent, which are deferred to Phase 2.

The substantive achievement of this paper is not merely a post-quantum transparent transaction format. It is the reconstruction of CryptoNote-style stealth ownership — static addresses, one-time spend keys, deterministic seed recovery, and nullifier-based double-spend prevention — using only standardized NIST post-quantum primitives, with no elliptic-curve cryptography anywhere in the validation path. This is the foundation, not the ceiling: future extensions (KDSK-CT for hidden amounts, KDSK-Ring for sender ambiguity) build on top without revisiting addresses, key derivation, scanning, or output formats.

1.1 Position in the Karbo Transaction Family

This paper specifies transaction family v3 — the post-quantum branch of the Karbo transaction system. v3 transactions coexist with two earlier families on the same chain:

- **v1 (legacy CryptoNote):** plain amounts, MLSAG ring signatures, Ed25519 stealth addresses. Existing chain history; continues to validate indefinitely.
- **v2 (Confidential Transactions):** Pedersen commitments, Groth-Kohlweiss one-of-many proofs over a 64-denomination set, MLSAG ring signatures, Ed25519 stealth addresses. Adds amount privacy on classical elliptic-curve security. Specified by the Karbo Hard Fork spec v1.3.2.
- **v3 (post-quantum, this paper):** ML-KEM-768 view keys, ML-DSA-65 per-output spend keys, plain amounts, no ring signatures. New address format. Quantum-resistant ownership and supply integrity.

The three families are independent: a node dispatches validation by transaction version byte. v2 and v3 solve orthogonal problems — v2 hides amounts under classical security, v3 hides nothing but moves ownership onto post-quantum cryptography. They are deployed in separate hard forks and users may hold balances in any combination.

1.2 What This Paper Cements

This paper is the v3 ownership and stealth layer. The following design elements are cemented and will not change in future v3 upgrades:

- **Address format** (PQAddressV1: version, network prefix, ML-KEM-768 view public key, checksum). The address a user holds today is permanent.
- **View key primitive** (ML-KEM-768). Every output ever sent to a v3 address uses ML-KEM encapsulation for delivery.
- **Wallet seed derivation chain** (`seed_master` → HKDF → `view_seed` → ML-KEM.KeyGen). Wallet recovery from seed always produces the same view keypair.

- **Per-output spend key derivation** (ss → HKDF → spend_seed → ML-DSA.KeyGen). The relationship between a received output and its spend key is permanent.
- **Output wire format** (PQOutputV1). Outputs created today can be referenced by any future spend mechanism without re-creation.

1.3 What May Extend Later

Spend-side mechanics — the input wire format, the signature scheme, the double-spend prevention mechanism — are open to future extension via new transaction subtypes within the v3 family. Section 17 describes two concrete directions: hidden amounts (v3-hidden) and post-quantum ring signatures (v3-ring). Both extensions read the same PQOutputV1 outputs specified here; only the input and balance-proof structures change.

2. Cryptographic Parameters (Normative)

Role	Primitive	FIPS	Key / output sizes
View / scan key	ML-KEM-768	FIPS 203	pk: 1184 B sk: 2400 B ct: 1088 B ss: 32 B
Per-output spend key	ML-DSA-65	FIPS 204	pk: 1952 B sk: 4032 B sig: 3309 B seed: 32 B
Hash	SHA3-256	FIPS 202	Output: 32 B
KDF	HKDF-SHA3-256	RFC 5869 + FIPS 202	salt: 32×0x00 L: 32 B (always)
AEAD	ChaCha20-Poly1305 (IETF)	RFC 8439	key: 32 B nonce: 12 B tag: 16 B
Backup signature	SLH-DSA	FIPS 205	Reserve — not used in Phase 1

HKDF instantiation. All HKDF calls in this document use HKDF-SHA3-256 with a fixed salt of 32 zero bytes (0x00×32) and output length L=32 bytes unless explicitly stated otherwise. The info string is the domain separation label shown in each derivation. Implementations **MUST NOT** use a different salt or output length.

ML-DSA seed. FIPS 204 ML-DSA.KeyGen takes a 32-byte seed (called xi in the standard). The spend_seed produced by HKDF is used directly as xi. No further hashing is applied.

ChaCha20-Poly1305 nonce. The nonce is 12 zero bytes. This is safe because the AEAD key is unique per output — derived from a per-output KEM shared secret. Key uniqueness makes nonce reuse harmless here.

3. Address Format

The PQ address contains only a view public key and a checksum. There is no auth root, epoch counter, or tree depth field.

```
PQAddressV1 = {  
  version      : uint8      // = 0x01  
  network_prefix : varint    // mainnet / testnet discriminator  
  view_pub_mlkem768 : bytes[1184] // ML-KEM-768 public key  
  checksum      : bytes[4]   // first 4 bytes of SHA3-256(version || prefix ||  
view_pub)  
}
```

```
Encoded: base58(version || network_prefix || view_pub_mlkem768 || checksum)  
Encoded length: ~1600 characters
```

```
QR code recommendation: use bech32m encoding with error correction level L  
to keep QR code density within scanner range for 1190-byte payload.
```

The address is static and reusable. Every output sent to this address produces a unique one-time spend key derived from the KEM session. No two outputs to the same address share a spend key, spend_commit, or nullifier, because each output has a unique KEM ciphertext and therefore a unique shared secret.

Account number registry — UX surface. The raw PQAddressV1 format is intended as the canonical cryptographic representation of an identity, not the primary user-facing surface. The base58-encoded address is approximately 1600 characters and is intended for wallet-to-wallet exchange in machine-readable form (QR codes, payment URIs, deep links). Deployments are expected to expose short user-facing account identifiers — registered on-chain via a separate account-number system — that resolve to PQAddressV1 records. Users transact by short account number; wallets resolve to raw addresses behind the scenes. This is the same separation between cryptographic identity and human-facing identifier used by ENS, Lightning addresses, and similar systems. The account number system is out of scope for this paper but assumed by the v3 deployment plan.

4. Deterministic Key Derivation from Seed

A wallet seed expands into two independent branches. The view branch produces the ML-KEM keypair embedded in the address. The spend branch is reserved for future use — per-output spend keys in this design come from KEM shared secrets, not from this branch.

```
seed_master (32 bytes, from BIP39 or equivalent CSPRNG)
```

```

view_seed = HKDF-SHA3-256(IKM=seed_master, salt=0x00*32,
                           info="karbo-pq-view-root-v1", L=32)

(view_pub, view_sk) = ML-KEM.KeyGen(seed=view_seed)
// view_pub is embedded in PQAddressV1
// view_sk is the wallet scanning key – kept secret

spend_root = HKDF-SHA3-256(IKM=seed_master, salt=0x00*32,
                           info="karbo-pq-spend-root-v1", L=32)
// reserved; not used in Phase 1 transaction validation

```

Wallet recovery from seed. Recovery requires a full chain rescan. For each PQ transaction on chain, the wallet computes `inputs_hash` from the transaction's input set, then for each output attempts `ML-KEM.Decaps(view_sk, kem_ct)`. If decapsulation yields a valid `ss`, the wallet recomputes `out_context` using `inputs_hash`, derives `aead_key`, and attempts AEAD decryption to recover `rho`. Amount is read directly from `PQOutputV1.amount` — it is not in the ciphertext. The spend keypair is re-derived from `ss` on demand at spend time. No per-output keys are stored persistently.

5. Per-Output Randomness (ρ)

ρ is 32 bytes of uniformly random data generated by the sender independently for each output. It must be drawn from a cryptographically secure random number generator.

ρ is included in the encrypted payload so the recipient can recover it. At spend time the spender reveals ρ in the transaction input. ρ binds the `spend_commit` and nullifier to this specific output, preventing a spender from substituting another output's credentials.

ρ reuse does not cause nullifier collision in this design. Because each output independently runs `ML-KEM.Encaps`, producing a fresh `kem_ct` and therefore a unique `ss` and unique `pk_i`, the nullifier = `SHA3-256(domain || pk_i || rho)` is unique per output even if ρ were reused. A CSPRNG is still required to ensure ρ is unpredictable, but reuse is not a structural threat.

Garbage output behaviour (inherent to stealth address systems)

A malicious sender can construct a `PQOutputV1` with a valid-length `kem_ct`, `enc_payload`, and `spend_commit` that are internally meaningless. The node accepts the transaction — all format checks pass. This is not a flaw specific to this design: it is an inherent property of any stealth address system, including the CryptoNote design Karbo uses today.

The recipient's wallet handles this correctly:

1. `ML-KEM.Decaps` runs on the garbage `kem_ct` and silently returns a pseudorandom `ss` — no error.

2. The AEAD key derived from that fake ss is wrong.
3. ChaCha20-Poly1305 decryption fails at tag authentication — the output is discarded immediately.
4. The output never appears in the wallet's balance. The user is unaware it exists.

Who is harmed: only the sender. The coins in the garbage output are burned — they came from the sender's inputs and go into an output nobody can ever spend. The receiver loses nothing and never had those coins. The only rational use of this attack is chain spam, which is deterred by the fee requirement.

This behaviour is identical to sending coins to an invalid address in any other cryptocurrency. The node cannot prevent it without access to the recipient's private view key, which would break the privacy model. Detection happens at wallet scan time, not spend time.

6. Output Creation

The sender encapsulates against the recipient's ML-KEM view key to establish a shared secret ss. All per-output key material is derived from ss using HKDF with distinct domain strings.

6.1 Output context and consensus-enforced uniqueness

out_context is a stable per-output identifier computed before any key derivation. It must be consensus-enforced unique — not merely assumed unique from honest sender behavior — to prevent a malicious sender from crafting two outputs with identical spend credentials.

The attack without a uniqueness guarantee: a malicious sender deliberately reuses the same kem_ct and output_index across two different transactions. Both transactions derive identical ss, out_context, pk_i, spend_commit, and nullifier. Both outputs may scan as owned by the recipient. Once one is spent its nullifier enters the set; the other output becomes permanently unspendable even though it is a distinct on-chain UTXO.

Fix: fold an inputs hash into out_context. Every transaction's inputs reference specific UTXOs via prev_txid || prev_out_index. These are guaranteed unique by existing UTXO consensus — the same input set cannot produce two confirmed transactions because the first spend invalidates those UTXOs. Therefore the inputs hash is a consensus-enforced uniqueness source that requires no new transaction fields.

```
// Step 1: compute inputs hash (covers all inputs in canonical order)
inputs_hash = SHA3-256(
  "karbo-pq-inputs-hash-v1" ||
  FOR EACH input (in order):
    prev_txid           || // bytes[32]
    LE32(prev_out_index) // 4 bytes
)
// inputs_hash: 32 bytes
```

```

    // Unique per confirmed transaction by UTX0 consensus.

// Step 2: encapsulate
(ss, kem_ct) = ML-KEM.Encaps(view_pub_mlkem768)
    // ss: 32-byte shared secret (ephemeral, never stored on chain)
    // kem_ct: 1088-byte ciphertext (stored in PQOutputV1)

// Step 3: compute output context
out_context = SHA3-256(
    "karbo-pq-out-context-v1" ||
    inputs_hash                || // 32 bytes – consensus-enforced unique
    kem_ct                    || // 1088 bytes
    LE32(output_index)        // 4 bytes, little-endian
)
    // out_context: 32 bytes
    // A malicious sender reusing kem_ct and output_index must also reuse
    // the same input set, which consensus makes impossible to confirm twice.

```

6.2 Key derivation from shared secret

```

// All HKDF calls: salt=0x00*32, L=32

spend_seed = HKDF-SHA3-256(
    IKM = ss,
    info = "karbo-pq-spend-seed-v1" || out_context
)
    // spend_seed: 32 bytes – used as ML-DSA.KeyGen seed (xi)

(pk_i, sk_i) = ML-DSA.KeyGen(xi = spend_seed)
    // pk_i: 1952 bytes – revealed only at spend time
    // sk_i: 4032 bytes – never stored; re-derived on demand

aead_key = HKDF-SHA3-256(
    IKM = ss,
    info = "karbo-pq-aead-key-v1" || out_context
)
    // aead_key: 32 bytes – ChaCha20-Poly1305 key

```

6.3 Commitments

spend_commit binds the output to a specific (pk_i, rho) pair. The node checks it at spend time to confirm the revealed pk_i and rho correspond to this output.

nullifier is derived from the same inputs as spend_commit but with a distinct domain string. The node recomputes it at spend time from the revealed auth_pub and rho_reveal. It is NEVER

serialized in the transaction — the node always derives it independently and never trusts a provided value.

Having two distinct commitments is intentional: `spend_commit` is checked against the output record, while the nullifier indexes into the anti-replay set. They serve different lookup purposes and must remain separate.

```
spend_commit = SHA3-256(
    "karbo-pq-spend-commit-v1" ||
    pk_i                        || // 1952 bytes
    rho                        // 32 bytes
)
// spend_commit: 32 bytes – stored in PQOutputV1, checked at spend time

// nullifier is NOT computed at output creation time.
// It is computed by the node at spend time from revealed fields.
// See Section 8.2 for the normative definition.
```

6.5 Payload encryption

The payload carries only rho. Amount is already on-chain in plaintext in `PQOutputV1.amount` — there is no reason to place it in the encrypted payload. Encrypting a value that is also plaintext in the same output creates two sources of truth with no benefit.

Instead, amount is bound into the AEAD authentication tag via additional data (AAD). AAD is authenticated but not encrypted — the Poly1305 tag cryptographically covers both the ciphertext (rho) and the AAD (`out_context || amount`). If anyone mutates `PQOutputV1.amount` after the sender created the payload, the tag becomes invalid and the recipient's decryption fails outright at the authentication step. The output becomes invisible to the wallet rather than being credited with a wrong amount. This gives the recipient hard cryptographic assurance that the plaintext amount was not tampered with in transit.

Since the plaintext is always exactly 32 bytes (rho only), the node can enforce a fixed `enc_payload` length at submission time.

```
enc_payload = ChaCha20-Poly1305.Encrypt(
    key   = aead_key,                // 32 bytes
    nonce = 0x00 * 12,              // 12 zero bytes – safe: key is unique per
    output
    aad   = out_context || LE64(amount), // 40 bytes – amount bound into tag, not
    ciphertext
    msg   = rho                      // 32 bytes – the only secret transmitted
)
// enc_payload: 32 (rho ciphertext) + 16 (Poly1305 tag) = 48 bytes

// amount is NOT in the plaintext.
```



```
// It is in AAD: authenticated, visible, but any mutation breaks the tag.  
// Consensus rule: reject enc_payload where length != 48 bytes.
```

6.6 Output wire structure

```
PQOutputV1 = {  
  type      : uint8      = 0x10 // PQ_PLAIN_V1 type tag  
  amount    : uint64      // consensus-authoritative; also authenticated  
via AEAD AAD  
  kem_ct    : bytes[1088] // ML-KEM-768 ciphertext  
  enc_payload : bytes[48] // ChaCha20-Poly1305(msg=rho, aad=out_context||  
amount)  
  spend_commit : bytes[32] // SHA3-256("karbo-pq-spend-commit-v1" || pk_i ||  
rho)  
}  
// total per output: 1 + 8 + 1088 + 48 + 32 = 1177 bytes
```

Important. `pk_i` is not present in the output. Only `spend_commit` appears on chain at creation time. The spend key is hidden until the output is spent.

7. Recipient Scanning and Wallet Recovery

Scanning mirrors CryptoNote's mental model: attempt decapsulation on every PQ output; success is confirmed by the `spend_commit` check.

5. Attempt `ML-KEM.Decaps(view_sk, kem_ct)`. ML-KEM always returns a value — a malformed `kem_ct` returns a pseudorandom ss rather than an error.
6. **Recompute out_context**: `SHA3-256("karbo-pq-out-context-v1" || inputs_hash || kem_ct || LE32(output_index))`. `inputs_hash` is derived from the transaction's input set — the wallet reads it from the transaction being scanned.
7. Derive `aead_key = HKDF-SHA3-256(ss, "karbo-pq-aead-key-v1" || out_context)`.
8. **Attempt ChaCha20-Poly1305.Decrypt**(key=aead_key, nonce=0x00*12, aad=out_context||LE64(amount), ct=enc_payload). If authentication fails, this output is not yours or the amount was tampered — discard.
9. Recover `rho` (bytes[32]) from decrypted payload. Amount is read directly from `PQOutputV1.amount` — it is not in the ciphertext.
10. Derive `spend_seed = HKDF-SHA3-256(ss, "karbo-pq-spend-seed-v1" || out_context)`. Compute `(pk_i, _) = ML-DSA.KeyGen(xi=spend_seed)`.
11. **Ownership check**: recompute `spend_commit = SHA3-256("karbo-pq-spend-commit-v1" || pk_i || rho)`. If it does not match `PQOutputV1.spend_commit`, discard — this output is not yours.

12. Store wallet record: { txid, output_index, amount, rho, out_context }. Do NOT store sk_i or spend_seed.

Step 7 is the critical ownership gate. An output must never be credited as spendable balance until spend_commit verification passes. Steps 4 and 7 together guarantee that only the holder of view_sk can claim ownership of an output.

8. Spending Path

8.1 Signing digest

The signing digest covers the complete unsigned transaction body. Implementations **MUST** serialize fields in the order specified and **MUST** include all fields listed.

```
tx_signing_digest = SHA3-256(
  "karbo-pq-tx-sign-v1" ||
  version                || // uint8
  LE32(num_inputs)       ||
  FOR EACH input (in order):
    prev_txid            || // bytes[32]
    LE32(prev_out_index) ||
    auth_pub             || // bytes[1952]
    rho_reveal           || // bytes[32]
  LE32(num_outputs)      ||
  FOR EACH output (in order):
    type                 || // uint8
    LE64(amount)         ||
    kem_ct               || // bytes[1088]
    enc_payload          || // bytes[48]
    spend_commit         || // bytes[32]
  LE64(fee)              // uint64
)
// tx_signing_digest: 32 bytes
// Nullifiers excluded – derived by node, not an input field.
// inputs_hash for out_context is derived from the same input fields above.
```

8.2 Nullifier (normative definition)

The nullifier is the sole anti-replay token for a PQ output. It is computed independently by every validating node from the fields revealed in the spending input. It is never serialized in the transaction wire format — the node always derives it and never trusts a provided value. This

eliminates a class of transaction malleability attacks where a mutated nullifier field could cause inconsistent validation.

```
nullifier = SHA3-256(
    "karbo-pq-nullifier-v1" ||
    pk_i                    || // bytes[1952] – auth_pub from input
    rho                    // bytes[32] – rho_reveal from input
)
// nullifier: 32 bytes
// Nodes MUST recompute from (auth_pub, rho_reveal) and reject
// any transaction where the recomputed value already exists
// in the nullifier set.
```

The nullifier is NOT serialized in PQInputV1. It does not appear on the wire.
Nodes compute it deterministically from auth_pub and rho_reveal.
This eliminates a class of transaction malleability attacks.

8.3 Spend key re-derivation

```
// Re-derive from stored wallet record { txid, output_index, amount, rho,
out_context }
// out_context is stored directly – no need to recompute inputs_hash at spend time
ss      = ML-KEM.Decaps(view_sk, kem_ct)
spend_seed = HKDF-SHA3-256(ss, "karbo-pq-spend-seed-v1" || out_context)
(pk_i, sk_i) = ML-DSA.KeyGen(xi = spend_seed)

// Sign
sig_msg = tx_signing_digest // see Section 8.1
signature = ML-DSA-65.Sign(sk_i, sig_msg)

// Discard sk_i immediately after signing
```

8.4 Input wire structure

```
PQInputV1 = {
    prev_txid      : bytes[32]
    prev_out_index : uint32
    auth_pub       : bytes[1952] // pk_i – revealed at spend time
    rho_reveal     : bytes[32]   // rho – revealed at spend time
    signature      : bytes[3309] // ML-DSA-65 signature over tx_signing_digest
}
// nullifier is NOT a field – nodes derive it
```

```
// total per input: 32 + 4 + 1952 + 32 + 3309 = 5329 bytes
```

8.5 Node validation rules (normative)

13. Verify referenced output exists and has type PQ_PLAIN_V1.
14. Recompute `spend_commit = SHA3-256("karbo-pq-spend-commit-v1" || auth_pub || rho_reveal)`. MUST equal `PQOutputV1.spend_commit` of the referenced output.
15. Recompute `nullifier = SHA3-256("karbo-pq-nullifier-v1" || auth_pub || rho_reveal)`. MUST NOT exist in the nullifier set.
16. Recompute `tx_signing_digest` per Section 8.1. Verify ML-DSA-65 signature using `auth_pub`.
17. Verify: `sum(PQOutputV1.amount for all inputs) == sum(PQOutputV1.amount for all outputs) + fee`.
18. Add nullifier to the nullifier set (keyed by block height for reorg support — see Section 11).

9. Transaction Type Taxonomy

Three transaction types are defined. Mixed PQ-input / legacy-output transactions are forbidden.

Type	Inputs	Outputs	Notes
TX_LEGACY_V1	Legacy CryptoNote	Legacy CryptoNote	Existing rules unchanged
TX_BRIDGE_V1	Legacy CryptoNote	PQOutputV1 only	One-way migration into PQ
TX_PQ_V1	PQInputV1 only	PQOutputV1 only	Pure PQ transaction

TX_BRIDGE_V1 validation. Validate legacy inputs under existing consensus rules. Enforce: `legacy_in_total == pq_out_total + fee`. No PQ inputs permitted in a bridge transaction.

TX_PQ_V1 validation. All inputs must be `PQInputV1`. All outputs must be `PQOutputV1`. Apply the rules in Section 8.5 to each input.

Fee handling. Fees are implicit in both `TX_BRIDGE_V1` and `TX_PQ_V1`, consistent with `CryptoNote` convention. The miner claims the difference between inputs and outputs as a fee reward in the coinbase. No explicit fee output is required. Miners do not need a PQ address to collect fees from PQ transactions.

Forbidden combinations: PQ inputs with legacy outputs are rejected by consensus. Legacy inputs with PQ inputs in the same transaction are rejected by consensus.

10. Transaction Size Budget

Field	Per unit	1-in / 2-out total
PQInputV1 (auth_pub + rho + sig + refs)	5,329 bytes	5,329 bytes
PQOutputV1 (kem_ct + payload + commit + amount + type)	1,177 bytes	2,354 bytes
Transaction overhead (version, counts, fee)	~20 bytes	~20 bytes
Total	—	~7,703 bytes

This is approximately 15× a typical legacy Karbo transaction. The following limits are consensus-enforced from day one:

- **Max PQ inputs per transaction:** 8
- **Max PQ outputs per transaction:** 16
- **Hard per-transaction size cap:** 48 KB
- **Minimum feerate for PQ transactions:** enforced in consensus (not relay policy only); value to be set by fork parameters in new atomic units per byte
- **Block size policy:** must be revisited independently — do not inherit legacy limits

11. Nullifier Set and Reorg Safety

The nullifier set is the sole anti-double-spend mechanism for PQ outputs. It must be persisted in LMDB from day one.

11.1 LMDB schema

```
Table: pq_nullifiers
Key:   nullifier (bytes[32])
Value: block_height (uint64) || txid (bytes[32])

Lookup: O(1) by nullifier hash
Rollback: DELETE WHERE block_height >= orphaned_height
```

11.2 Reorg procedure

When a block is orphaned, all nullifiers added at that block's height must be removed from the set before the competing chain tip is applied. The `block_height` stored per nullifier enables efficient rollback without scanning the full set.

19. Identify the fork point (last common ancestor block).
20. For each orphaned block from tip down to fork point: `DELETE FROM pq_nullifiers WHERE block_height == orphaned_block_height`.
21. Apply the competing chain forward from the fork point, re-adding nullifiers at each block.

The existing Karbo reorg limit of 10 blocks bounds the rollback depth. The nullifier set size added per rollback is therefore bounded by $10 \times \text{max_inputs_per_block}$.

12. Domain Separation — Normative Reference

All domain strings are UTF-8 encoded and included as raw bytes with no length prefix. No two derivations share a domain string.

Derivation	Domain string	HKDF IKM
View seed	karbo-pq-view-root-v1	seed_master
Spend root seed (reserved)	karbo-pq-spend-root-v1	seed_master
Inputs hash	karbo-pq-inputs-hash-v1	n/a — SHA3-256 not HKDF
Output context	karbo-pq-out-context-v1	n/a — SHA3-256 not HKDF
Per-output spend seed	karbo-pq-spend-seed-v1	ss
AEAD key	karbo-pq-aead-key-v1	ss
spend_commit	karbo-pq-spend-commit-v1	n/a — SHA3-256 not HKDF
nullifier	karbo-pq-nullifier-v1	n/a — SHA3-256 not HKDF
Transaction signing digest	karbo-pq-tx-sign-v1	n/a — SHA3-256 not HKDF

Test vectors for every derivation in this table **MUST** be published before any consensus code is merged. A mismatch in `spend_commit` or `nullifier` derivation between implementations is a chain split.

Reserved future namespace. Domain strings beginning with `karbo-pq-ct-` are reserved for the KDSK-CT extension (Section 17.1) and **MUST NOT** be used by v3-plain implementations. The first such reserved string is `karbo-pq-ct-mask-v1`, which derives commitment opening

randomness from the per-output KEM shared secret. Future additions under this prefix may be specified in the KDSK-CT companion paper.

13. Unspendable Output Attack Analysis

Attack	Detectable by node?	Detectable by wallet?	Mitigation
Malformed kem_ct length	Yes — at submission	N/A	Consensus: reject kem_ct != 1088 bytes
Garbage kem_ct or enc_payload (valid length)	No — node cannot decrypt	Yes — AEAD tag fails at scan step 4; output discarded	Inherent to stealth addresses. Sender burns own coins. Receiver unaffected.
Amount tampered in transit	No	Yes — AEAD tag fails (amount is in AAD)	Hard discard at scan step 4. Receiver unaffected.
kem_ct reused across two transactions	Yes — inputs_hash differs per confirmed tx	N/A	out_context binds to inputs_hash; same kem_ct in a different tx produces different out_context and different pk_i.
rho reuse (same tx, different output)	N/A — not a threat	N/A	Each output has unique kem_ct and therefore unique ss and pk_i; nullifier collision infeasible.
Nullifier pre-collision	No — computationally infeasible	N/A	SHA3-256 preimage resistance.

In all garbage-output cases the receiver is unaffected — the output never appears in their balance. The only victim is the sender who burned their own coins. This is an inherent property of stealth address systems, not a flaw in this design. The spend_commit check at scanning step 7 provides a final ownership gate for any case where AEAD decryption succeeds but key derivation produces wrong results.

14. Legacy Output Migration

Legacy Karbo outputs use plain transparent amounts. Migration into the PQ transaction family requires no commitment translation, range proofs, or amount-reveal cryptography.

14.1 TX_BRIDGE_V1 validation

Step 1: Validate all legacy CryptoNote inputs under existing consensus rules.
Step 2: legacy_in_total = sum of all legacy input amounts
 pq_out_total = sum of all PQOutputV1.amount fields
Step 3: Enforce: $\text{legacy_in_total} == \text{pq_out_total} + \text{fee}$
Step 4: Validate each PQOutputV1 format (type tag, field sizes).
Step 5: No PQ inputs permitted – reject if any PQInputV1 present.

14.2 Migration rules

- Legacy → PQ: allowed via TX_BRIDGE_V1.
- PQ → Legacy: forbidden at consensus level.
- Legacy spending remains valid indefinitely until an optional future sunset fork.
- Wallets must display PQ and legacy balances separately.
- Wallets must provide a dedicated migration function that constructs TX_BRIDGE_V1.

15. Recommended Implementation Order

22. Implement all HKDF derivations, out_context, spend_seed, aead_key, spend_commit, nullifier, and tx_signing_digest. Produce test vectors for each. Fuzz against known-answer tests before proceeding.
23. Integrate ML-KEM-768 and ML-DSA-65 from an audited library (e.g. liboqs). Do not hand-roll arithmetic. Verify encap/decap roundtrip and sign/verify roundtrip with known test vectors.
24. Implement address encoding: $\text{view_pub} \rightarrow \text{PQAddressV1} \rightarrow \text{base58/bech32m}$. Implement wallet import/export.
25. Implement PQOutputV1 creation (sender side): encaps, out_context, key derivation, AEAD encrypt, spend_commit.
26. Implement wallet scanning: decaps, out_context, AEAD decrypt, integrity check, spend_commit verify.
27. Implement LMDB nullifier table with block_height indexing. Implement rollback procedure.
28. Implement TX_PQ_V1 input validation: spend_commit check, nullifier recomputation and set lookup, ML-DSA signature verify, balance equation.
29. Implement TX_BRIDGE_V1 validation.
30. Implement spend path in wallet: spend key re-derivation, signing digest construction, ML-DSA sign.
31. Only after the above is stable: prototype PQ-private transaction families as a separate research track.

16. Security and Engineering Recommendations

- Use canonical transaction serialization per Section 8.1 before hashing and signing. Field order is normative.
- Use constant-time, audited implementations of ML-KEM and ML-DSA. Do not hand-roll arithmetic.
- Perform pairwise consistency checks (encap/decap, sign/verify) when generating keypairs where the library's guidance requires it.
- Do not derive any PQ key from an Ed25519 or legacy CryptoNote key. Use only fresh seeds from a CSPRNG.
- Treat `out_context` derivation as wallet-critical code — a bug produces outputs the wallet can scan but never spend.
- `sk_i` must be derived on demand and discarded immediately after signing. Do not persist it.
- Wallets must never credit an output as spendable until `spend_commit` verification (scanning step 8) passes.
- Do not ship ring signatures or hidden amounts in Phase 1.

17. Future Expansion Possibilities

v3 ships as plain-amount, single-signer, with public spend linkability. This is a deliberate Phase-1 trade-off that delivers quantum-resistant ownership without waiting on more advanced cryptographic research. The ownership layer specified in this paper is structured to allow two natural future extensions, both of which preserve the address format, view key, and output structure unchanged.

17.1 Hidden Amounts — KDSK Confidential Transactions (v3-hidden)

Amounts in v3 are currently plain. A future **v3-hidden** upgrade — also referred to as *KDSK Confidential Transactions (KDSK-CT)* — replaces the plaintext amount field with a lattice-based commitment to a denomination index, together with a denomination membership proof. The construction mirrors the structure of v2 CT (Pedersen commitment + Groth-Kohlweiss one-of-many proof) but uses quantum-safe lattice cryptography throughout.

KDSK-CT preserves the denomination-based model of v2 CT. Each output commits to exactly one value from the shared 64-denomination set spanning 10^{10} to 10^{17} atomic units. The proof system therefore proves **denomination membership** — that the committed index lies in the fixed 64-element set — rather than an arbitrary 64-bit range relation. This substantially reduces proof complexity compared to generic post-quantum range proofs and is, in current understanding, the only realistic route toward compact PQ confidential amounts.

The construction uses additively homomorphic lattice commitments (Module-LWE Ajtai construction). Validators verify the balance directly as a public commitment equation:

$$\sum C_{\text{inputs}} - \sum C_{\text{outputs}} = \text{PublicValue}(\text{fee})$$

Denomination membership proofs accompany each output, establishing that every committed value belongs to the fixed v2/v3 denomination set. A small auxiliary short-vector proof bounds the accumulated commitment randomness to prevent supply inflation through noise growth. The balance check itself is a direct algebraic equation in the commitment space, not a zero-knowledge proof — exactly as in v2 CT.

Fees remain transparent in KDSK-CT. No fee commitment is serialized. The transparent fee participates as a public term in the homomorphic commitment equation above. This simplification mirrors the v1 and v2 fee model and keeps mining incentives and fee policy unchanged at the consensus layer.

KDSK-CT is the subject of a separate companion paper (v3 PQ-CT Companion). It is research-stage and gated on production-grade lattice proof libraries reaching maturity. Estimated horizon is 12–24 months after v3 base layer deployment. KDSK-CT activates via its own hard fork and does not modify anything specified in this paper.

Compatibility property: Outputs created under this v3 specification can be spent into v3-hidden outputs in the same transaction. The PQOutputV1 format does not change — only a new output subtype is added alongside it. Wallets may hold mixed v3-plain and v3-hidden balances under the same address.

17.2 Sender Ambiguity — KDSK Ring Spends (v3-ring)

Spending in v3 currently reveals `auth_pub` and `rho_reveal` in the input, making the spend-to-output link public on chain. A future v3-ring upgrade can add a ring-signature input format where the spender selects N decoy outputs from chain history and proves knowledge of the spend key for one of them without revealing which.

Candidate constructions include lattice ring signatures (MatRiCT-class, ~30-50 KB at moderate ring sizes), Falcon-based one-of-many proofs, or MLSAG-style structures rebuilt over lattice commitments. The double-spend mechanism would change form: a key-image-like linker derived from (pk_i, sk_i) replaces the nullifier-as-hash-of-revealed-fields construction. Anti-replay semantics stay the same.

Compatibility property: PQOutputV1 outputs created under this paper are spendable by a future v3-ring input format without modification. The ring is constructed by selecting decoy outputs from existing chain history. A new transaction type — `TX_V3_RING` — coexists alongside `TX_V3` (current) and `TX_V3_HIDDEN` (future). The spender chooses which mechanism to use; the output does not care.

One honest limitation: Decoy anonymity ultimately relies on the long-term secrecy of view keys. A future quantum-broken ML-KEM could in principle let an attacker decapsulate recorded `kem_ct` values and correlate decoys to addresses retroactively. This is the same limitation any stealth address system faces. The anonymity guarantee in v3-ring is therefore classical-period only, similar to current Monero. Spend authorization remains quantum-safe regardless because it rests on ML-DSA-65, not ML-KEM.

17.3 Combined KDSK-CT + KDSK-Ring

v3-hidden and v3-ring are independent and can ship in either order or together. A combined transaction type — hidden amounts plus ring-signature inputs — gives the closest post-quantum analogue to current Monero's RingCT, with the caveats noted above on decoy anonymity under

future quantum break. Specification of a combined type is deferred until both component upgrades are individually mature.

17.4 Why This Foundation Supports All of the Above

The v3 ownership layer cleanly separates two concerns. Ownership — who can spend an output — is fixed at output creation via `spend_commit` binding `pk_i` and `rho`. Spend mechanics — how the spender proves authorization — is determined at spend time by the input format, which is independent of the output. This abstraction boundary is exactly the right place to extend the protocol:

- **New signature schemes** (ring, threshold, multi-sig PQ) plug in as new input wire formats over existing outputs.
- **New amount representations** (hidden via commitments, optionally including range or set proofs) plug in as new output subtypes alongside `PQOutputV1`.
- **Address format and view scanning** are unchanged by any of the above. The user-facing identity surface is permanent.

Implementers and reviewers of v3 should treat this paper as the foundation, not the ceiling. Future extensions build on top; they do not require revisiting addresses, keys, scanning, or output formats.

17.5 Shared KEM Session Derivation (Reserved Namespace)

Per this v3 specification (Section 6), the ML-KEM-768 shared secret `ss` derived from each output is used to produce `spend_seed` (via the domain string `karbo-pq-spend-seed-v1`) and `aead_key` (via `karbo-pq-aead-key-v1`). This pattern — independent HKDF derivations from the same per-output KEM session, separated by explicit domain strings — extends naturally to future amount-hiding and commitment-opening material.

To enable a clean future upgrade, the following derivation namespace is **reserved** by this paper and **MUST NOT** be used in v3 plain implementations:

```
mask_seed = HKDF-SHA3-256(  
    IKM    = ss,  
    info   = "karbo-pq-ct-mask-v1" || out_context,  
    L      = 32  
)
```

Future KDSK-CT outputs derive their commitment blinding randomness and denomination opening material from `mask_seed` via further domain-separated HKDF expansions. The architectural significance: ownership, scanning, and commitment opening all originate from the same ML-KEM stealth-delivery session. A single decapsulation reproduces the full set of per-output secrets a wallet needs.

This reservation is normative for v3 implementations to the following extent: the domain string `karbo-pq-ct-mask-v1` **MUST NOT** be used by any v3-plain operation. Other domain strings beginning with `karbo-pq-ct-` are similarly reserved for future KDSK-CT use and **MUST NOT** be repurposed.

Appendix A — Complete Wire Format Summary

```
// Address
PQAddressV1 = {
    version          : uint8          // 0x01
    network_prefix   : varint
    view_pub_mlkem768 : bytes[1184]
    checksum         : bytes[4]
} // ~1190 bytes before encoding

// Output
PQOutputV1 = {
    type            : uint8          // 0x10 = PQ_PLAIN_V1
    amount          : uint64         // consensus-authoritative
    kem_ct          : bytes[1088]
    enc_payload     : bytes[48] // ChaCha20-Poly1305(msg=rho[32], aad=out_context||
amount)
    spend_commit    : bytes[32]
} // 1177 bytes

// Input
PQInputV1 = {
    prev_txid       : bytes[32]
    prev_out_index  : uint32
    auth_pub        : bytes[1952] // ML-DSA-65 public key
    rho_reveal      : bytes[32]
    signature       : bytes[3309] // ML-DSA-65 signature
} // 5329 bytes (nullifier NOT serialized – node derives it)
```

Appendix B — Standards Basis

Standard	Primitive	Status	Usage in this proposal
FIPS 203	ML-KEM (Kyber)	Published	ML-KEM-768: view keypair, output scanning
FIPS 204	ML-DSA (Dilithium)	Published	ML-DSA-65: per-output spend keypair
FIPS 202	SHA-3	Published	SHA3-256 for all hashing; HKDF-SHA3-256 for KDF
RFC 8439	ChaCha20-Poly1305	Published	AEAD encryption of output payload
RFC 5869	HKDF	Published	Key derivation framework

Standard	Primitive	Status	Usage in this proposal
FIPS 205	SLH-DSA (SPHINCS+)	Published	Reserve path — not used in Phase 1
FIPS 206	FN-DSA (Falcon)	Draft	Not used — standard not finalized

Appendix C — Changes from v2.0

Issue	v2.0 state	v2.1 resolution
out_context definition	Two conflicting definitions (body vs patch)	Single normative definition: SHA3-256(domain kem_ct LE32(output_index)). tx_prefix_hash removed to eliminate circular dependency.
Nullifier serialization	Contradicted between Section 7.1 and patch	Nullifier is NOT serialized. Node always recomputes from auth_pub and rho_reveal.
tx_prefix_hash circular dependency	out_context depended on tx_prefix_hash which depended on spend_commit which depended on out_context	Resolved by removing tx_prefix_hash from out_context. kem_ct provides uniqueness instead.
rho generation	Never specified	Normatively specified: 32 bytes CSPRNG, sender-generated, unique per output.
HKDF parameters	Info strings only; salt and L unspecified	Fully specified: salt=0x00*32, L=32 for all calls.
AEAD cipher	Never named	ChaCha20-Poly1305 (IETF, RFC 8439).
spend_commit vs nullifier rationale	Unexplained duplication	Explained: serve different lookup purposes (output record vs anti-replay set).
Amount authoritative source	Ambiguous with encrypted copy	Plaintext amount is consensus-authoritative; encrypted copy is wallet integrity check only.
Signing digest contents	"canonical_tx_bytes" undefined	Section 8.1 specifies exact field order and all included fields.
LMDB nullifier reorg	Not addressed	Section 11: block_height stored per nullifier; rollback procedure specified.
TX type taxonomy	TX_BRIDGE_V1 only; PQ-in/PQ-out unnamed	Three types defined: TX_LEGACY_V1, TX_BRIDGE_V1, TX_PQ_V1.
Fee handling	Not specified for PQ transactions	Fees are implicit (CryptoNote convention); miners claim via coinbase.
seed_spend unused	Derived but never referenced again	Marked as reserved for Phase 2; not used in Phase 1 validation.
Implementation list	Started at 8	Fixed — starts at 1.

Issue	v2.0 state	v2.1 resolution
numbering		
QR code / address size	Not addressed	Recommendation added: bech32m with error correction level L.